

Projet application ArtConnect

Bases de données Avancées – M. Zebiri

Arnaud Sekhara, Roy Tamwo, Hippolyte Vallat, Fahed Tadrist

Projet ArtConnect

Livrable 3 : Implémentation de la Base de Données et Fonctionnalités Avancées

Arnaud Sekhara, Roy Tamwo,
Hippolyte Vallat, Fahed Tadrist

Le 10/05/2026

Année scolaire 2025 – 2026

1. Insertion de données

Des données réalistes et cohérentes ont été insérées dans chacune des tables principales afin de tester les relations, les contraintes d'intégrité et les fonctionnalités avancées de la base de données ArtConnect.

1.1 Utilisation d'un LLM pour la génération des données

Conformément à la consigne, un LLM (**Claude 3.5 Sonnet**) a été utilisé pour générer les données d'exemple. Le prompt soumis ainsi que le script produit ont été conservés et vérifiés manuellement avant intégration.

Prompt soumis à Claude (LLM) :

```
Tu es un expert en bases de données MySQL. Génère un script SQL
d'insertion de données
d'exemple réalistes pour une application culturelle appelée ArtConnect.

La base contient les tables suivantes :
- Artist (id_artist, name, bio, birthYear, contactEmail, phone, city,
website,
socialMedia, isActive)
- Artwork (id_artwork, title, creationYear, type, medium, dimensions,
description,
price, status, id_artist)
- Gallery (id_gallery, name, street_number, street_name, country,
city, zip_code,
ownerName, openingHours, contactPhone, rating, website)
- Exhibition (id_exhibition, title, startDate, endDate, description,
curatorName,
theme, id_gallery)
- Workshop (id_workshop, title, date, durationMinutes,
maxParticipants, price,
location, description, level, id_artist)
- CommunityMember (id_member, name, email, birthYear, phone, city,
membershipType)
- Booking (id_booking, bookingDate, paymentStatus, id_member,
id_workshop)
- Discipline (id_discipline, name)
- ArtworkTag (id_artworkTag, name)

Contraintes et cas intéressants à couvrir :
- Au moins 5 artistes avec des disciplines variées (peinture,
sculpture, photographie,
street art, céramique)
- Au moins 8 oeuvres dont plusieurs du même artiste, avec statuts
variés
(AVAILABLE, SOLD, ON_LOAN)
- Au moins 3 galeries dans des villes différentes
- Au moins 4 expositions dont certaines dans la même galerie à des
dates différentes
- Au moins 5 workshops dont certains animés par le même artiste, avec
niveaux variés
(BEGINNER, INTERMEDIATE, ADVANCED)
- Au moins 6 membres avec types d'abonnement variés (FREE, PREMIUM,
ARTIST)
- Des réservations croisées : un même membre réserve plusieurs
workshops,
```

- un même workshop a plusieurs participants
- Des statuts de paiement variés (PAID, PENDING, CANCELLED)
- Des disciplines et tags cohérents avec les artistes et oeuvres

Génère uniquement le script SQL (INSERT INTO ...) sans CREATE TABLE.
Utilise des données françaises réalistes (noms, villes, adresses).
Assure la cohérence des clés étrangères.

Le script généré a été relu et testé manuellement sur la base de données avant validation. Aucune incohérence de clé étrangère n'a été détectée. Les statuts, niveaux et types d'abonnement respectent les contraintes ENUM définies lors de la création des tables (Livrable 2).

1.2 Tables alimentées

Les tables suivantes ont été peuplées avec des données cohérentes :

- Artist : 5 artistes avec disciplines variées (peinture, sculpture, photographie, street art, céramique)
- Artwork : 8 oeuvres réparties entre les artistes, avec statuts AVAILABLE, SOLD et ON_LOAN
- Gallery : 3 galeries dans Paris, Lyon et Bordeaux
- Exhibition : 4 expositions dont deux hébergées par la même galerie à des dates distinctes
- Workshop : 5 ateliers animés par différents artistes avec niveaux BEGINNER, INTERMEDIATE et ADVANCED
- CommunityMember : 6 membres avec abonnements FREE, PREMIUM et ARTIST
- Booking : réservations croisées (un membre sur plusieurs workshops, plusieurs membres sur un même workshop)
- Discipline & ArtworkTag : données de référence cohérentes

1.3 Extraits du script SQL généré et vérifié

Insertion des artistes :

```
INSERT INTO Artist (name, bio, birthYear, contactEmail, phone, city, website,
isActive) VALUES
('Sophie Martin', 'Peintre contemporaine, spécialisée en art abstrait.',
1985, 'sophie.martin@artmail.fr', '0612345678', 'Paris',
'www.sophiemartin.fr', TRUE),
('Julien Renard', 'Sculpteur travaillant le métal et le bois recyclé.',
1978, 'julien.renard@artmail.fr', '0623456789', 'Lyon',
'www.julienrenard.fr', TRUE),
('Camille Dubois', 'Photographe documentaire, voyages et portraits urbains.',
1990, 'camille.dubois@artmail.fr', '0634567890', 'Bordeaux',
'www.camilledubois.fr', TRUE),
('Karim Benali', 'Artiste street art, murales engagées en milieu urbain.',
1995, 'karim.benali@artmail.fr', '0645678901', 'Marseille',
'www.karimbenali.fr', TRUE),
('Elise Fontaine', 'Céramiste, création de pièces uniques à la main.',
1983, 'elise.fontaine@artmail.fr', '0656789012', 'Nantes',
'www.elisefontaine.fr', TRUE);
```

Insertion des oeuvres d'art (extraits) :

```
INSERT INTO Artwork (title, creationYear, type, medium, dimensions,
                    description, price, status, id_artist) VALUES
('Lumières d automne', 2022, 'Peinture', 'Huile sur toile', '80x100cm',
 'Composition abstraite aux tons chauds.', 1800.00, 'AVAILABLE', 1),
('Fragment urbain', 2021, 'Sculpture', 'Métal recyclé', '50x30x20cm',
 'Assemblage de pièces métalliques trouvées.', 2500.00, 'SOLD', 2),
('Regards croisés', 2023, 'Photo', 'Tirage argentique', '40x60cm',
 'Série de portraits en noir et blanc.', 950.00, 'AVAILABLE', 3),
('Mur de la mémoire', 2020, 'Street art', 'Bombe aérosol', '300x200cm',
 'Murale réalisée pour la Mairie de Marseille.', 3200.00, 'ON_LOAN', 4),
('Vase aux racines', 2024, 'Céramique', 'Grès émaillé', '35x20cm',
 'Pièce unique inspirée des formes organiques.', 680.00, 'AVAILABLE', 5);
```

Réservations croisées (participations multiples) :

```
-- Membre 1 réserve 3 workshops différents
INSERT INTO Booking (bookingDate, paymentStatus, id_member, id_workshop) VALUES
(NOW(), 'PAID', 1, 1),
(NOW(), 'PAID', 1, 2),
(NOW(), 'PENDING', 1, 3),
-- Plusieurs membres réservent le même workshop
(NOW(), 'PAID', 2, 1),
(NOW(), 'PAID', 3, 1),
(NOW(), 'CANCELLED', 4, 2),
(NOW(), 'PAID', 5, 3),
(NOW(), 'PENDING', 6, 4),
(NOW(), 'PAID', 2, 5);
```

2. Vues SQL

Trois vues ont été créées afin de simplifier les requêtes complexes, améliorer la lisibilité du code applicatif et présenter les informations essentielles de manière structurée.

Nom de la vue	Description	Tables impliquées
vue_artworks_par_artiste	Liste des oeuvres avec nom de l'artiste, type, prix et statut	Artwork, Artist
vue_reservations_workshops	Détail des réservations : membre, workshop, date, paiement	Booking, CommunityMember, Workshop
vue_expositions_publicques	Expositions en cours ou à venir avec galerie, dates et thème	Exhibition, Gallery

2.1 Vue : artworks par artiste

```
CREATE VIEW vue_artworks_par_artiste AS
SELECT ar.title AS titre_oeuvre,
       a.name AS nom_artiste,
       ar.type,
       ar.price AS prix,
       ar.status AS statut
FROM Artwork ar
JOIN Artist a ON ar.id_artist = a.id_artist;
```

2.2 Vue : réservations de workshops

```
CREATE VIEW vue_reservations_workshops AS
SELECT cm.name      AS membre,
       w.title      AS atelier,
       b.bookingDate AS date_reservation,
       b.paymentStatus AS statut_paiement
FROM Booking b
JOIN CommunityMember cm ON b.id_member = cm.id_member
JOIN Workshop w        ON b.id_workshop = w.id_workshop;
```

2.3 Vue : expositions publiques

```
CREATE VIEW vue_expositions_publiques AS
SELECT e.title      AS exposition,
       g.name       AS galerie,
       e.startDate  AS debut,
       e.endDate    AS fin,
       e.theme
FROM Exhibition e
JOIN Gallery g ON e.id_gallery = g.id_gallery
WHERE e.endDate >= CURDATE();
```

3. Index

Des index ont été ajoutés sur les colonnes les plus fréquemment utilisées dans les filtres WHERE, les jointures et les tris ORDER BY, afin d'optimiser les performances des requêtes.

Table	Colonne indexée	Justification
Artist	contactEmail	Recherche fréquente d'artistes par email
Artist	city	Filtrage par ville pour les événements locaux
Artwork	id_artist	Clé étrangère utilisée dans de nombreuses jointures
Artwork	status	Filtrage des oeuvres disponibles à la vente
Workshop	date	Tri et filtrage par date pour l'agenda
Booking	id_member	Récupération rapide des réservations d'un membre
Booking	id_workshop	Comptage des participants par workshop
Exhibition	id_gallery	Jointure avec la galerie hôte

3.1 Création des index

```
CREATE INDEX idx_artist_email ON Artist(contactEmail);
CREATE INDEX idx_artist_city ON Artist(city);
CREATE INDEX idx_artwork_status ON Artwork(status);
CREATE INDEX idx_artwork_artist ON Artwork(id_artist);
```

```
CREATE INDEX idx_workshop_date      ON Workshop(date);
CREATE INDEX idx_booking_member     ON Booking(id_member);
CREATE INDEX idx_booking_workshop   ON Booking(id_workshop);
CREATE INDEX idx_exhibition_gallery ON Exhibition(id_gallery);
-- Index composite pour l'analyse des réservations
CREATE INDEX idx_booking_member_ws  ON Booking(id_member, id_workshop);
```

4. Triggers

Trois triggers ont été mis en place afin d'automatiser des règles métier critiques et de garantir l'intégrité des données sans intervention de l'application cliente.

4.1 Vérification de la capacité maximale des workshops

Ce trigger s'exécute avant chaque INSERT dans la table Booking. Il vérifie que le nombre de réservations existantes pour un workshop ne dépasse pas sa capacité maximale. En cas de dépassement, une erreur SQL est levée.

```
DELIMITER $$
CREATE TRIGGER trg_check_workshop_capacity
BEFORE INSERT ON Booking
FOR EACH ROW
BEGIN
    DECLARE nb_participants INT;
    DECLARE max_cap INT;
    SELECT COUNT(*) INTO nb_participants
        FROM Booking WHERE id_workshop = NEW.id_workshop;
    SELECT maxParticipants INTO max_cap
        FROM Workshop WHERE id_workshop = NEW.id_workshop;
    IF nb_participants >= max_cap THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Capacité maximale du workshop atteinte.';
    END IF;
END$$
DELIMITER ;
```

4.2 Vérification des dates d'exposition

Ce trigger s'exécute avant chaque INSERT dans la table Exhibition et vérifie que la date de fin est bien postérieure à la date de début.

```
DELIMITER $$
CREATE TRIGGER trg_check_exhibition_dates
BEFORE INSERT ON Exhibition
FOR EACH ROW
BEGIN
    IF NEW.endDate <= NEW.startDate THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'La date de fin doit etre postérieure à la date de
début.';
    END IF;
END$$
DELIMITER ;
```

4.3 Audit des changements de prix des oeuvres

Ce trigger enregistre dans une table d'audit tout changement de prix d'une oeuvre, avec l'ancien prix, le nouveau prix et l'horodatage.

```
CREATE TABLE ArtworkPriceAudit (  
  id_audit INT AUTO_INCREMENT PRIMARY KEY,  
  id_artwork INT NOT NULL,  
  old_price DECIMAL(10,2),  
  new_price DECIMAL(10,2),  
  changed_at DATETIME DEFAULT NOW()  
);  
  
DELIMITER $$  
CREATE TRIGGER trg_audit_artwork_price  
AFTER UPDATE ON Artwork  
FOR EACH ROW  
BEGIN  
  IF OLD.price <> NEW.price THEN  
    INSERT INTO ArtworkPriceAudit (id_artwork, old_price, new_price)  
    VALUES (NEW.id_artwork, OLD.price, NEW.price);  
  END IF;  
END$$  
DELIMITER ;
```

5. Procédures stockées et fonctions

Des procédures stockées et une fonction SQL encapsulent les traitements métier récurrents afin de simplifier les appels depuis l'application.

5.1 Procédure : réserver un workshop

```
DELIMITER $$  
CREATE PROCEDURE sp_reserver_workshop(  
  IN p_id_member INT,  
  IN p_id_workshop INT  
)  
BEGIN  
  INSERT INTO Booking (bookingDate, paymentStatus, id_member, id_workshop)  
  VALUES (NOW(), 'PENDING', p_id_member, p_id_workshop);  
END$$  
DELIMITER ;  
  
-- Appel  
CALL sp_reserver_workshop(3, 2);
```

5.2 Procédure : ajouter une oeuvre d'art

```
DELIMITER $$  
CREATE PROCEDURE sp_ajouter_artwork(  
  IN p_title VARCHAR(200),  
  IN p_year INT,  
  IN p_type VARCHAR(100),  
  IN p_price DECIMAL(10,2),  
  IN p_id_artist INT  
)  
BEGIN  
  INSERT INTO Artwork (title, creationYear, type, price, status, id_artist)
```

```
VALUES (p_title, p_year, p_type, p_price, 'AVAILABLE', p_id_artist);
END$$
DELIMITER ;

-- Appel
CALL sp_ajouter_artwork('Lumière d automne', 2025, 'Peinture', 1200.00, 1);
```

5.3 Fonction : nombre de participants à un workshop

```
DELIMITER $$
CREATE FUNCTION fn_nb_participants(p_id_workshop INT)
RETURNS INT
DETERMINISTIC
BEGIN
    DECLARE nb INT;
    SELECT COUNT(*) INTO nb FROM Booking WHERE id_workshop = p_id_workshop;
    RETURN nb;
END$$
DELIMITER ;

-- Utilisation
SELECT fn_nb_participants(2) AS nombre_de_participants;
```

6. Transactions

Les transactions SQL garantissent l'atomicité des opérations critiques : soit toutes les instructions s'exécutent avec succès, soit aucune modification n'est appliquée.

6.1 Réservations multiples simultanées

```
START TRANSACTION;

INSERT INTO Booking (bookingDate, paymentStatus, id_workshop, id_member)
VALUES (NOW(), 'PAID', 2, 1);

INSERT INTO Booking (bookingDate, paymentStatus, id_workshop, id_member)
VALUES (NOW(), 'PAID', 3, 1);

COMMIT;
```

6.2 Gestion des erreurs avec ROLLBACK

```
DELIMITER $$
CREATE PROCEDURE sp_double_reservation(
    IN p_id_member INT, IN p_ws1 INT, IN p_ws2 INT
)
BEGIN
    DECLARE EXIT HANDLER FOR SQLEXCEPTION
    BEGIN
        ROLLBACK;
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Echec de la double réservation. Annulation.';
    END;
    START TRANSACTION;
    INSERT INTO Booking (bookingDate, paymentStatus, id_member, id_workshop)
    VALUES (NOW(), 'PAID', p_id_member, p_ws1);
```



```
INSERT INTO Booking (bookingDate, paymentStatus, id_member, id_workshop)
VALUES (NOW(), 'PAID', p_id_member, p_ws2);
COMMIT;
END$$
DELIMITER ;
```

7. Requêtes avancées de test

Des requêtes SQL ont été rédigées afin de valider le bon fonctionnement des vues, triggers, procédures et index mis en place.

7.1 Vérification des vues

```
SELECT * FROM vue_artworks_par_artiste;
SELECT * FROM vue_reservations_workshops;
SELECT * FROM vue_expositions_publiques WHERE debut <= NOW();
```

7.2 Vérification des triggers

```
-- Tester le trigger de capacité (doit lever une erreur si workshop plein)
INSERT INTO Booking (bookingDate, paymentStatus, id_workshop, id_member)
VALUES (NOW(), 'PAID', 1, 99);

-- Vérifier l'audit des prix
UPDATE Artwork SET price = 1500.00 WHERE id_artwork = 1;
SELECT * FROM ArtworkPriceAudit;
```

7.3 Statistiques et analyses

```
-- Nombre de participants par workshop
SELECT w.title,
       fn_nb_participants(w.id_workshop) AS participants,
       w.maxParticipants AS capacite_max
FROM Workshop w;

-- Revenus par workshop (paiements confirmés)
SELECT w.title,
       COUNT(b.id_booking) AS nb_reservations,
       SUM(w.price) AS revenu_total
FROM Workshop w
JOIN Booking b ON w.id_workshop = b.id_workshop
WHERE b.paymentStatus = 'PAID'
GROUP BY w.id_workshop, w.title;
```

Conclusion

L'étape 3 du projet ArtConnect a permis de concrétiser l'implémentation complète de la base de données relationnelle MySQL, en s'appuyant sur la modélisation conceptuelle et logique des livrables précédents.

Les données d'exemple ont été générées à l'aide du LLM Claude, avec conservation du prompt et vérification manuelle du script produit. Les fonctionnalités avancées mises en oeuvre garantissent :

- L'intégrité des données via les triggers (contrôle de capacité, validation des dates, traçabilité des prix)
- La performance des requêtes grâce aux index ciblés sur les colonnes critiques
- La lisibilité et la maintenabilité du code via les vues SQL
- La réutilisabilité de la logique métier dans les procédures stockées et fonctions
- La fiabilité des opérations groupées grâce aux transactions avec gestion des erreurs

Le script SQL complet de l'étape 3 est fourni dans le fichier ArtConnect_Etape3_Final.sql.